

Home Assignment – Week 2/Day 1

1. Instructions:

- Fund transfer test cases within the same bank
- [STRICT] Functional only — positive and negative
- [MANDATORY] Include TC ID, Priority, Description, Precondition, Steps, Expected/Actual Result, Test Data
- [CRITICAL] Mix of P0, P1, P2
- [MANDATORY] Exactly 10 test cases
- Then switch persona and review, adding comments

Context:

Banking app's fund transfer feature — transfers between two accounts within the same bank.

Example:

TC-ID: TC001 Priority: P1

Description: Verify fund transfer between two same-bank accounts succeeds

Precondition: App running; both accounts have sufficient balance

Steps: 1) Log in 2) Enter destination account 3) Enter amount & submit

Expected Result: Transfer succeeds; both balances update

Test Data: Acc 1: 445556 · Acc 2: 434343 · Amount: \$1000

Persona:

- Generate: Banking Domain QA Engineer, expert in functional testing
- Review: Team Lead, reviewing the test cases

Output Format:

10 test cases in a table, followed by a short review summary and Team Lead comments.

Tone:

[STRICT] Professional tone in review comments and summary.

TC ID	Priority	Description	Precondition	Steps	Expected / Actual Result	Test Data
TC001	P0	Verify fund transfer between two same-bank accounts succeeds with sufficient balance	User is logged in; source and destination accounts are active; source has sufficient balance	1. Select Fund Transfer 2. Select source account 3. Enter valid same-bank destination account 4. Enter valid amount 5. Submit transfer 6. Confirm transaction	Expected: Transfer succeeds; confirmation/reference is displayed; source balance decreases and destination balance increases by the transfer amount. Actual: TBD during execution	Source: 445556 Destination: 434343 Amount: \$1,000
		Verify transfer is rejected when source account has insufficient funds	User is logged in; source account is active; source balance is less than transfer amount	1. Select Fund Transfer 2. Enter valid same-bank destination 3. Enter amount greater than source balance 4. Submit transfer	Expected: Transfer is rejected with an insufficient-funds message; no balance is debited; destination is not credited. Actual: TBD during execution	Source balance: \$500 Destination: 434343 Amount: \$1,000
TC003	P0	Verify transfer to an invalid	User is logged in; source	1. Select Fund Transfer	Expected: Transfer is rejected and an	Source: 445556

TC ID	Priority	Description	Precondition	Steps	Expected / Actual Result	Test Data
TC004	P1	or non-existent same-bank account is rejected	account has sufficient balance	2. Enter invalid/non-existent destination account 3. Enter valid amount 4. Submit transfer	appropriate invalid-account message is displayed; source balance remains unchanged. Actual: TBD during execution	Destination: 999999 Amount: \$500
		Verify transfer of a valid minimum supported amount succeeds	User is logged in; both accounts are active; source has sufficient balance; minimum transfer limit is configured	1. Select Fund Transfer 2. Enter valid destination 3. Enter minimum allowed amount 4. Submit transfer	Expected: Transfer succeeds for the configured minimum amount and both account balances are updated correctly. Actual: TBD during execution	Source: 445556 Destination: 434343 Amount: Minimum allowed amount
		Verify transfer exceeding the configured maximum transaction limit is rejected	User is logged in; both accounts are active; maximum transfer limit is configured	1. Select Fund Transfer 2. Enter valid destination 3. Enter amount above maximum allowed limit 4. Submit transfer	Expected: Transfer is rejected with an appropriate limit-validation message; no funds are transferred. Actual: TBD during execution	Source: 445556 Destination: 434343 Amount: Maximum limit + \$1
TC006	P1	Verify zero-value fund transfer is rejected	User is logged in; both accounts are active	1. Select Fund Transfer 2. Enter valid destination 3. Enter transfer amount as 0 4. Submit transfer	Expected: Transfer is rejected with an appropriate amount-validation message; account balances remain unchanged. Actual: TBD during execution	Source: 445556 Destination: 434343 Amount: \$0
TC007	P1	Verify transfer with a negative amount is rejected	User is logged in; both accounts are active	1. Select Fund Transfer 2. Enter valid destination 3. Enter a negative transfer amount 4. Submit transfer	Expected: Negative amount is rejected through validation; no transfer is processed and balances remain unchanged. Actual: TBD during execution	Source: 445556 Destination: 434343 Amount: -\$100
TC008	P2	Verify transfer to the same source account is rejected	User is logged in; source account is active and has sufficient balance	1. Select Fund Transfer 2. Select source account 3. Enter the same account as destination 4. Enter valid amount 5. Submit transfer	Expected: Transfer is rejected with an appropriate same-account validation message; source balance remains unchanged. Actual: TBD during execution	Source: 445556 Destination: 445556 Amount: \$100
TC009	P2	Verify transfer with a blank destination	User is logged in; source account is	1. Select Fund Transfer 2. Leave destination account blank	Expected: Transfer is rejected with a mandatory-field	Source: 445556 Destination: Blank Amount: \$500

TC ID	Priority	Description	Precondition	Steps	Expected / Actual Result	Test Data
TC010	P2	account is rejected	active and has sufficient balance	3. Enter valid transfer amount 4. Submit transfer	validation message; no funds are transferred. Actual: TBD during execution Expected: Transfer is rejected with a mandatory amount-validation message; account balances remain unchanged. Actual: TBD during execution	
		Verify transfer with a blank amount is rejected	User is logged in; source and destination accounts are active	1. Select Fund Transfer 2. Enter valid same-bank destination 3. Leave amount blank 4. Submit transfer		Source: 445556 Destination: 434343 Amount: Blank

2. Instructions:

- Generate 5 sample user records for testing
- [STRICT] Include realistic positive and edge-case data
- [MANDATORY] Each record must contain: User ID, First Name, Last Name, Email, Phone Number, Country, Date of Birth, and Account Status
- [CRITICAL] Include edge cases such as special characters, very long names, international phone numbers, and boundary values
- [MANDATORY] Ensure every record is unique and uses valid field formats
- Do not include explanations outside the requested output

Context:

A user registration form used by an international application. The QA team needs representative test data to validate field handling, internationalization, input validation, and edge cases.

Example:

User ID: USR-1001
First Name: Priya
Last Name: Sharma
Email: priya.sharma@example.com
Phone Number: +91 98765 43210
Country: India
Date of Birth: 1995-06-15
Account Status: Active

Persona:

- Generate: Test Data Generator focused on creating realistic and comprehensive QA test data
- Review: QA Engineer reviewing data quality, uniqueness, field validity, and edge-case coverage

Output Format:

JSON array containing exactly 5 user records with the following fields:

User ID, First Name, Last Name, Email, Phone Number, Country, Date of Birth, Account Status

Tone:

[STRICT] Concise, data only — no explanations

```
[
{
  "User ID": "USR-1001",
```

```

"First Name": "Priya",
"Last Name": "Sharma",
"Email": "priya.sharma@example.com",
"Phone Number": "+91 98765 43210",
"Country": "India",
"Date of Birth": "1995-06-15",
"Account Status": "Active"
},
{
"User ID": "USR-1002",
"First Name": "Zoë",
"Last Name": "O'Connor",
"Email": "zoe.oconnor@example.com",
"Phone Number": "+44 7700 900123",
"Country": "United Kingdom",
"Date of Birth": "1988-02-29",
"Account Status": "Active"
},
{
"User ID": "USR-1003",
"First Name": "MaximilianAlexanderChristopher",
"Last Name": "Van Der Meer-Søndergaard",
"Email": "maximilian.vanderm@sample.example",
"Phone Number": "+31 6 12345678",
"Country": "Netherlands",
"Date of Birth": "1970-01-01",
"Account Status": "Suspended"
},
{
"User ID": "USR-1004",
"First Name": "李",
"Last Name": "王",
"Email": "li.wang@example.cn",
"Phone Number": "+86 138 0013 8000",
"Country": "China",
"Date of Birth": "2008-09-12",
"Account Status": "Pending"
},
{
"User ID": "USR-1005",
"First Name": "Amina",
"Last Name": "El-Haddad",
"Email": "amina.el-haddad@example.ae",
"Phone Number": "+971 50 123 4567",
"Country": "United Arab Emirates",
"Date of Birth": "1900-01-01",
"Account Status": "Inactive"
}
]

```

3. Instructions:

- Identify the top 10 highest-risk regression test cases to run first from a provided list of 50 test cases

- [STRICT] Prioritize tests based on business/module criticality, recent defect history, failure impact, and likelihood of regression
- [MANDATORY] Assign a risk score to every selected test case
- [CRITICAL] Rank the 10 selected test cases from highest to lowest risk
- [MANDATORY] Provide a clear justification for the ranking of each selected test case
- [CRITICAL] Ensure the prioritization considers both business impact and technical regression risk
- Do not modify or rewrite the original test case descriptions
- Do not include tests outside the provided list of 50 test cases

Context:

A QA team has a list of 50 existing test cases that need to be executed during regression testing.

Due to limited regression testing time, the team needs to identify the 10 highest-risk test cases that should be executed first.

The test cases cover multiple application modules with different levels of business criticality and varying histories of recent defects.

Example:

Test Case: TC-042

Module: Payment

Risk Score: 95

Priority Rank: 1

Justification: Payment is a business-critical module with multiple recent defects, making this test case highly likely to expose a regression with significant customer impact.

Persona:

- Generate: QA Automation Lead focused on regression testing, risk-based test prioritization, automation coverage, business criticality, and defect history
- Review: QA Team Lead reviewing the prioritization logic, risk scores, ranking consistency, and regression coverage

Output Format:

Ranked table containing exactly 10 test cases with the following fields:

Rank, TC ID, Module, Risk Score, Business Criticality, Recent Defect History, Regression Risk, Justification

Follow the table with a short review containing:

Review Summary:

- Prioritization Quality
- Risk Coverage
- Identified Gaps

Team Lead Comments:

- Comments on ranking consistency, risk-score justification, and any missing high-risk scenarios

Tone:

[STRICT] Concise, technical, risk-focused — no unnecessary explanations

4. Instructions:

- Identify the top 10 highest-risk regression test cases to run first from a provided list of 50 test cases
- [STRICT] Prioritize tests based on business/module criticality, recent defect history, failure impact, and likelihood of regression
- [MANDATORY] Assign a risk score to every selected test case
- [CRITICAL] Rank the 10 selected test cases from highest to lowest risk
- [MANDATORY] Provide a clear justification for the ranking of each selected test case
- [CRITICAL] Ensure the prioritization considers both business impact and technical regression risk
- Do not modify or rewrite the original test case descriptions
- Do not include tests outside the provided list of 50 test cases

Context:

A QA team has a list of 50 existing test cases that need to be executed during regression testing.

Due to limited regression testing time, the team needs to identify the 10 highest-risk test cases that should be executed first.

The test cases cover multiple application modules with different levels of business criticality and varying histories of recent defects.

Example:

Test Case: TC-042

Module: Payment

Risk Score: 95

Priority Rank: 1

Justification: Payment is a business-critical module with multiple recent defects, making this test case highly likely to expose a regression with significant customer impact.

Persona:

- Generate: QA Automation Lead focused on regression testing, risk-based test prioritization, automation coverage, business criticality, and defect history
- Review: QA Team Lead reviewing the prioritization logic, risk scores, ranking consistency, and regression coverage

Output Format:

Ranked table containing exactly 10 test cases with the following fields:

Rank, TC ID, Module, Risk Score, Business Criticality, Recent Defect History, Regression Risk, Justification

Follow the table with a short review containing:

Review Summary:

- Prioritization Quality
- Risk Coverage
- Identified Gaps

Team Lead Comments:

- Comments on ranking consistency, risk-score justification, and any missing high-risk scenarios

Tone:

[STRICT] Concise, technical, risk-focused — no unnecessary explanations

For the test cases, refer to the 1st item in the same doc – table

Rank	TC ID	Module	Risk Score	Business Criticality	Recent Defect History	Regression Risk	Justification
1	TC001	Fund Transfer	95	Critical	Not provided	Very High	Core fund-transfer success path. Failure directly blocks a critical banking transaction and can cause incorrect debit/credit behaviour. High technical regression exposure because it exercises the complete transfer flow and balance updates.
2	TC002	Fund Transfer	92	Critical	Not provided	Very High	Validates the critical insufficient-funds control. A regression could permit unauthorized overdrawn or incorrect balance changes, creating direct financial impact.
3	TC003	Fund Transfer	90	Critical	Not provided	Very High	Invalid destination-account handling is a key transaction-integrity control. A defect could result in funds being routed incorrectly or transaction validation being bypassed.
4	TC005	Fund Transfer	86	High	Not provided	High	Maximum transaction-limit enforcement protects against excessive transfers and compliance/business-rule violations. Boundary validation is technically prone to regression after transaction-rule changes.
5	TC004	Fund Transfer	82	High	Not provided	High	Minimum-amount processing validates an important transaction boundary while also exercising successful

Rank	TC ID	Module	Risk Score	Business Criticality	Recent Defect History	Regression Risk	Justification
6	TC008	Fund Transfer	75	High	Not provided	Medium-High	debit/credit processing. Boundary-value logic has meaningful regression risk. Prevents transfers to the source account itself. Incorrect handling could create invalid ledger behavior or transaction-processing issues, although customer impact is lower than core transfer failures.
7	TC006	Fund Transfer	68	Medium-High	Not provided	Medium	Zero-value validation protects transaction integrity and prevents invalid transactions. Important negative coverage, but business impact is lower than insufficient-funds or account-validation failures.
8	TC007	Fund Transfer	66	Medium-High	Not provided	Medium	Negative-amount validation protects against invalid financial input and potential calculation defects. Technical risk exists, but the scenario is less likely to affect normal production transactions.
9	TC009	Fund Transfer	60	Medium	Not provided	Medium	Blank destination validation protects mandatory transaction input. Failure would generally prevent or disrupt a transaction rather than cause incorrect financial movement.
10	TC010	Fund Transfer	58	Medium	Not provided	Medium	Blank amount validation is important input validation, but failure is primarily a transaction-blocking issue with lower financial impact than successful-transfer and balance-integrity scenarios.

Review Summary:

- **Prioritization Quality:** Strong risk-based ordering using business impact, transaction criticality, financial consequences, and technical regression exposure. Core transaction and financial-control scenarios are ranked first.
- **Risk Coverage:** Covers successful transfer, financial validation, account validation, transaction limits, boundary conditions, and invalid-input scenarios.
- **Identified Gaps:** Recent defect history was **not provided**, so it could not be used to differentiate the scores. Only TC001–TC010 were supplied; therefore, no unsupported TC IDs from the remaining 40 cases were selected.

Team Lead Comments:

- **Ranking consistency:** Consistent; higher ranks correspond to greater potential financial/business impact and broader transaction-flow coverage.
- **Risk-score justification:** Scores reflect relative risk rather than test priority alone. TC001–TC003 receive the highest scores because failures could directly affect transaction execution or financial integrity.
- **Missing high-risk scenarios:** The remaining 40 test cases are required to determine whether any scenarios have higher risk than TC001–TC010, particularly cases involving authentication/authorization, duplicate transactions, concurrency, rollback, transaction timeout/retry, audit logging, or recent production defects.
- **Defect-history limitation:** No recent defect data was supplied. If defect history is added, the scores and ordering should be recalibrated accordingly.